


```
|         |─ views/          # 视图
|         |─ components/     # 组件
|         |─ services/       # API 接口
|         |─ stores/         # 存储
|─ deploy/nginx/             # Nginx 配置
|─ docker-compose.yml        # Docker 配置
|─ ai_web_service.sh         # 启动脚本
```

3. Docker Compose

3.1 克隆

```
git clone git@github.com:know-nothing0302/ai_web.git
cd ai_web
git checkout main
```

3.2 配置

```
cp apps/api/.env.example apps/api/.env
# 编辑 .env 文件
```

配置内容:

```
# 数据库
POSTGRES_HOST=your-pg-host
POSTGRES_PORT=5432
POSTGRES_USER=postgres
POSTGRES_PASSWORD=your-password
POSTGRES_DB=ai_web

# Session
SESSION_SECRET=your-random-secret

# CAS 配置
CAS_LOGIN_URL=https://cas.xzhmu.edu.cn/login
CAS_VALIDATE_URL=https://cas.xzhmu.edu.cn/serviceValidate
CAS_SERVICE_URL=http://your-domain/api/auth/cas/callback
CAS_LOGOUT_URL=https://cas.xzhmu.edu.cn/logout

# DeepSeek API
DEEPSEEK_API_BASE_URL=https://api.deepseek.com
DEEPSEEK_API_KEY=sk-your-key
DEEPSEEK_MODEL=deepseek-chat
```

3.3 启动

```
docker compose up
```

访问:

- API: <http://localhost:3000>
- Web: <http://localhost:5173>

- Nginx: <http://localhost:8080>

3.4 部署

```
# 部署 API
npm run dev:api      # tsx watch 3000

# 部署 HMR
npm run dev:web      # Vite 5173
```

4. 部署

4.1 环境

- IP: 172.30.4.45 (xyd-45)
- 系统: Ubuntu 22.04 LTS
- 发行版: ubuntu
- 路径: /opt/idapps/ai_web

4.2 部署

```
# 1. 克隆
cd /opt
git clone git@github.com:know-nothing0302/ai_web.git
cd ai_web

# 2. 安装依赖
cd apps/api && npm install --include=dev && cd ../..

# 3. 配置环境变量
cp apps/api/.env.example apps/api/.env
vim apps/api/.env      # 配置环境变量

# 4. 构建
npm run build:api

# 5. 创建 TS 脚本
mkdir -p apps/api/dist/scripts
cp apps/api/image/ apps/api/dist/image/ -r
cp apps/api/src/scripts/gen_birthday_card.py apps/api/dist/scripts/

# 6. 配置 systemd
sudo cp /opt/idapps/ai_web/deploy/systemd/idapps-ai-web.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable idapps-ai-web
sudo systemctl start idapps-ai-web
```

4.3 Systemd 配置

配置 systemd 服务 `node dist/server.js` 为 `ai_web_service.sh` 执行

```
# 启动
sudo systemctl start idapps-ai-web
```

```
# 检查
sudo systemctl stop idapps-ai-web

# 检查
sudo systemctl restart idapps-ai-web

# 检查
sudo systemctl status idapps-ai-web

# 检查
sudo journalctl -u idapps-ai-web -f          # 查看
sudo journalctl -u idapps-ai-web --since today # 查看
```

4.4 部署

```
# 部署
git push origin feature/xxx

# 部署
cd /opt/idapps/ai_web
git pull origin feature/xxx
npm run build:api
sudo systemctl restart idapps-ai-web

# 测试
curl http://127.0.0.1:3000/api/health
```

注意: PSD 文件通过 git 同步到 .gitignore 文件中

```
scp apps/api/image/*.psd xyd-45:/opt/idapps/ai_web/apps/api/image/
```

5. Nginx 配置

5.1 Docker Compose 配置

```
server {
    listen 80;
    server_name _;

    location /api/ {
        proxy_pass http://api:3000/api/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    location / {
        proxy_pass http://web:5173/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
    }
}
```

5.2 配置

```
server {
    listen 80;
    server_name ai.xzhu.edu.cn;

    # 静态资源
    location /ai-web/ {
        alias /opt/idapps/ai_web/apps/web/dist/;
        try_files $uri $uri/ /ai-web/index.html;
    }

    # API 代理
    location /api/ {
        proxy_pass http://127.0.0.1:3000/api/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

6. 部署

6.1 Schema 初始化

Schema 初始化——数据库初始化 db.ts 中 schemaSql 中 ALTER TABLE ... ADD COLUMN IF NOT EXISTS

```
// apps/api/src/lib/db.ts
// 数据库初始化
await pool.query(schemaSql); // 表 + 索引
await pool.query(seedSql); // 数据
```

6.2 数据表

表名	说明
users	用户信息 Oracle 表
articles	文章
channels	频道/列表
page_agent_conversations	PageAgent 会话
page_agent_messages	PageAgent 消息
subscriptions	订阅
feedback	反馈
user_profiles	用户档案

6.3 部署

```
# 备份脚本
pg_dump -U postgres ai_web > /backup/ai_web_$(date +%Y%m%d).sql

# 7 天后删除
find /backup/ -name "ai_web_*.sql" -mtime +7 -delete
```

7. 环境变量配置

变量名	值	说明
NODE_ENV	development	运行环境
PORT	3000	API 端口
SESSION_SECRET	-	Session 密钥 (必填)
APP_BASE_URL	http://localhost:3000	API 基 URL
WEB_BASE_URL	http://localhost:5173	前端 URL
CAS_LOGIN_URL	-	CAS 登录 URL
CAS_VALIDATE_URL	-	CAS 验证 URL
CAS_SERVICE_URL	-	CAS 服务 URL
CAS_LOGOUT_URL	-	CAS 退出 URL
DEV_AUTH_BYPASS	true	开发环境跳过认证
POSTGRES_HOST	localhost	数据库主机
POSTGRES_PORT	5432	数据库端口
POSTGRES_USER	postgres	数据库用户
POSTGRES_PASSWORD	-	数据库密码
POSTGRES_DB	ai_web	数据库名
DEEPSEEK_API_BASE_URL	-	DeepSeek API 基 URL
DEEPSEEK_API_KEY	-	DeepSeek API 密钥
DEEPSEEK_MODEL	deepseek-chat	模型名称
DEEPSEEK_USER_ID_SECRET	-	用户 ID 密钥
PAGE_AGENT_DEBUG	true	PageAgent 调试
WECOM_CORP_ID	-	企业 ID
WECOM_AGENT_ID	-	企业微信 AgentID
WECOM_SECRET	-	企业微信 Secret
WECOM_PUSH_AGENT_ID	-	推送 AgentID
WECOM_PUSH_SECRET	-	推送 Secret
WECOM_INTERNAL_AUTH_TOKEN	-	内部 API 认证 Token

ADMIN_USER_IDS	-	データベース
ORACLE_USER	-	Oracle データベース
ORACLE_PASSWORD	-	Oracle パスワード
ORACLE_CONNECT_STRING	-	Oracle 接続文字列
DAILY_PUSH_CRON	0 20 * * *	毎日 cron
DAILY_PUSH_CRON_2	0 11 * * *	毎日 cron
WEEKLY_PUSH_CRON	0 20 * * 0	毎週 cron
PROFILE_ANALYSIS_CRON	0 3 * * 0	毎週 cron
BIRTHDAY_PUSH_MODE	test	テストモード (test/production)

8. 確認

確認

```
# 1. 確認
pgrep -f "node dist/server.js"

# 2. 確認
ss -tlnp | grep 3000

# 3. 確認
curl http://127.0.0.1:3000/api/health
# 返: {"ok":true,"service":"ai_web_api"}

# 4. CAS 確認
# データベース → CAS → 認証 → 認可
# ネットワーク 確認 401/403

# 5. PageAgent 確認
# データベース → AI 処理 → 認証 → 認可

# 6. 確認
# データベース → 認証 → 認可
```

9. 確認

確認

```
# 確認
sudo journalctl -u idapps-ai-web -n 50 --no-pager

# 確認
psql -h $POSTGRES_HOST -U $POSTGRES_USER -d $POSTGRES_DB -c "SELECT 1"

# 確認
ss -tlnp | grep 3000
```

PageAgent 测试

```
# 测试 DeepSeek API 测试
curl -H "Authorization: Bearer $DEEPSEEK_API_KEY" \
    $DEEPSEEK_API_BASE_URL/v1/models

# 测试 PageAgent 测试
sudo journalctl -u idapps-ai-web | grep "page.agent"
```

测试步骤

```
# 获取 Token
# 测试部署
sudo journalctl -u idapps-ai-web | grep "push"
```